

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

PENGANTAR MIKROKONTROLER

Reidita Eirene Anastasia Katampuge - 5024241001

2026

1 Langkah-Langkah Percobaan

1.1 Percobaan 1: Pengenalan STM32 Blackpill (*Blink*)

1. Mengisi `platformio.ini` dengan *platform* `ststm32`, *board* `blackpill_f401cc`, *framework* `arduino`, serta `upload_protocol` dan `debug_tool` bernilai `stlink`.

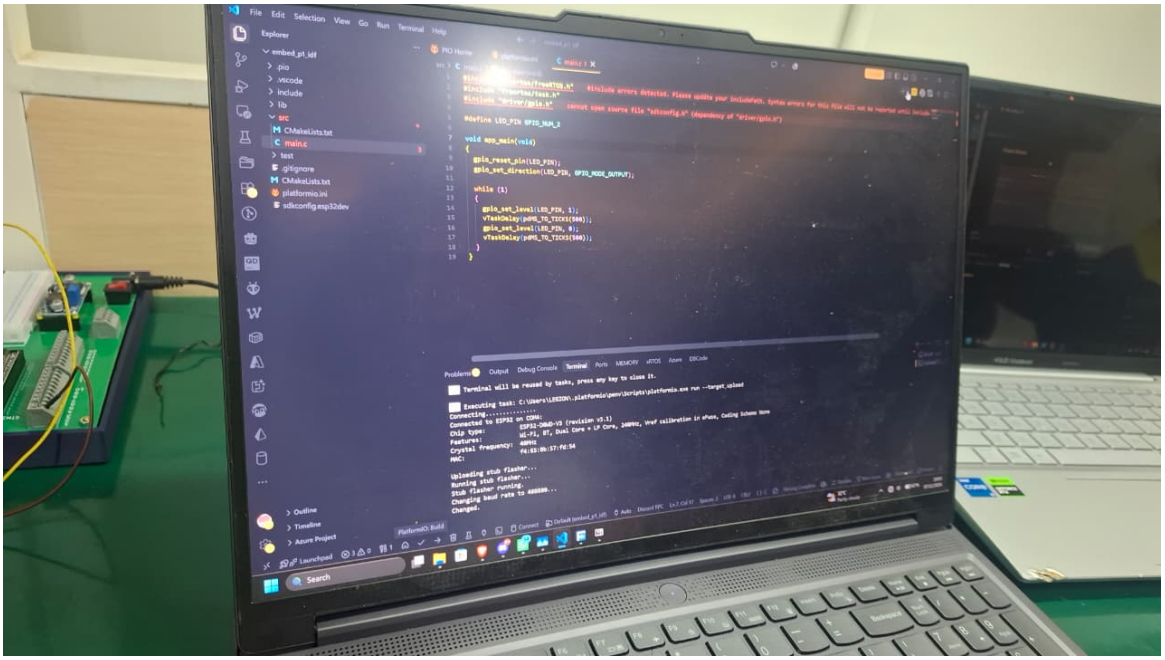
2. Menghubungkan ST-Link ke Blackpill dengan kabel jumper sesuai pasangan pin SWDIO ke SWDIO, SWCLK ke SWCLK, GND ke GND, dan 3.3V ke 3V3, dengan memastikan ST-Link memakai tegangan 3,3 V. Setelah itu, ST-Link dihubungkan ke komputer melalui kabel USB.



Gambar 1: Hubungkan ST-Link ke Blackpill lalu ST-Link dihubungkan ke komputer

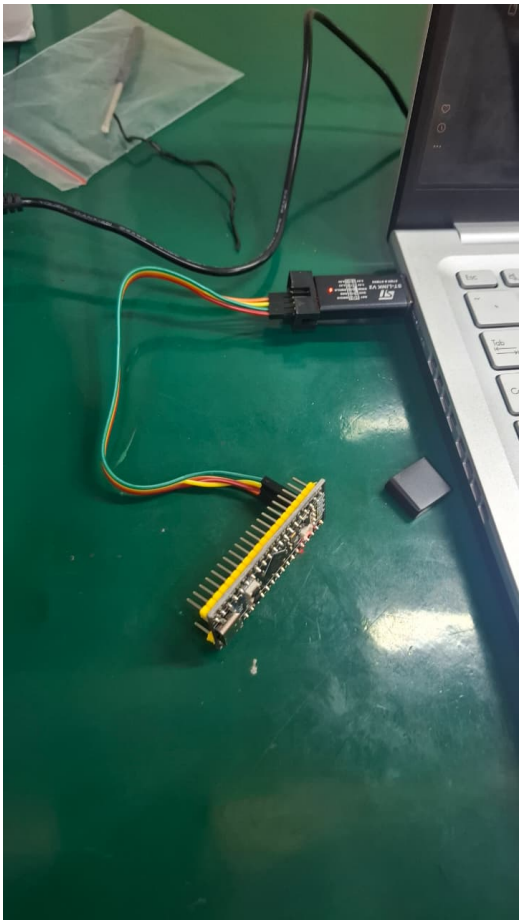
3. Menulis program *Blink* pada `src/main.cpp` yang mengatur pin PC13 sebagai keluaran dan menyalakan serta mematikan LED setiap 500 ms. LED *onboard* bersifat aktif *LOW*.

4. Menekan *Build* lalu *Upload*.



Gambar 2: Menekan Build dan Upload

5. Mengamati LED *onboard* pada pin PC13, yang seharusnya menyala 0,5 detik dan mati 0,5 detik secara berulang.



Gambar 3: Amati LED

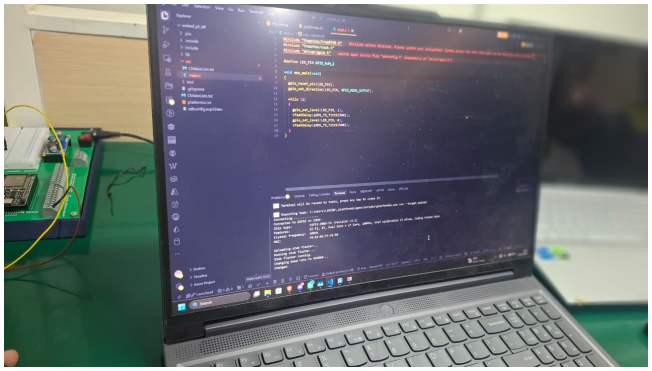
1.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (*Blink*)

1. Mengganti isi `platformio.ini` dengan *platform* `espressif32`, *board* `esp32dev`, dan *framework* `espidf`, lalu menunggu proses *refresh* selesai.
2. Menghubungkan ESP32 ke komputer melalui kabel USB dan memastikan port-nya dikenali.



Gambar 4: Hubungkan ESP32 ke Komputer

3. Menghapus berkas `src/main.cpp`, kemudian membuat berkas baru `src/main.c`.
4. Menulis program *Blink* dengan fungsi `app_main()` yang mengatur GPIO 2 sebagai keluaran dan mengubah levelnya setiap 500 ms menggunakan `vTaskDelay(pdMS_TO_TICKS(500))`.
5. Menekan *Build* lalu *Upload*



Gambar 5: Build Lalu Upload

6. Mengamati LED onboard ESP32.

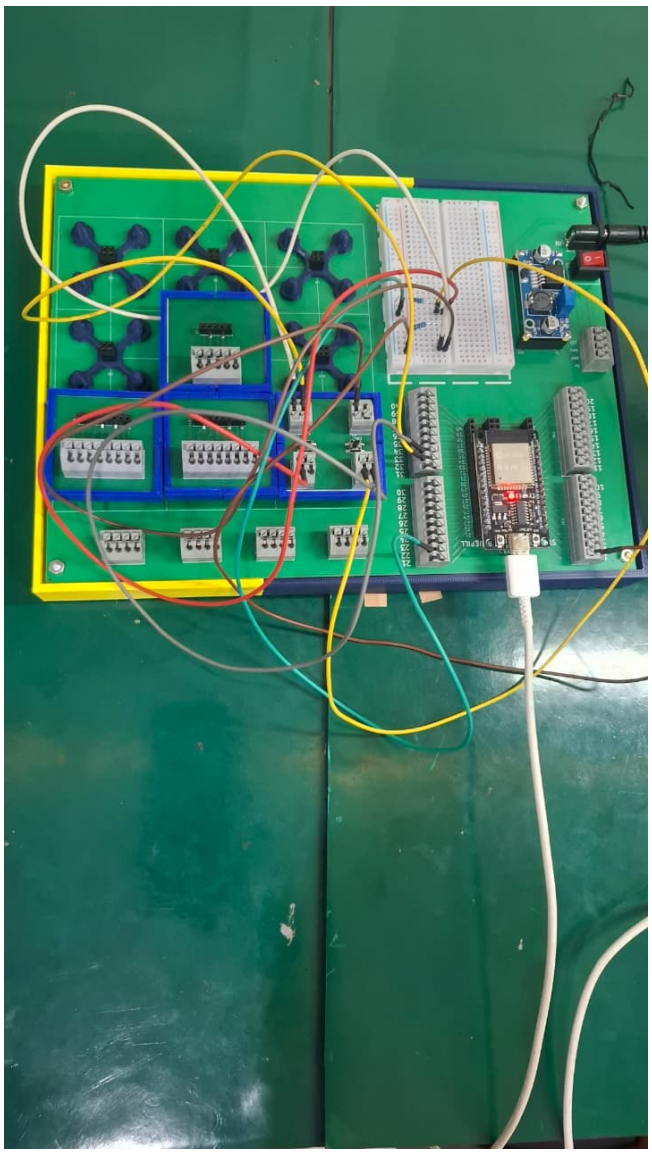


Gambar 6: Mengamati LED ESP32

1.3 Percobaan 3: *Pull-up* dan *Pull-down* Eksternal dan Internal

Bagian A: resistor eksternal

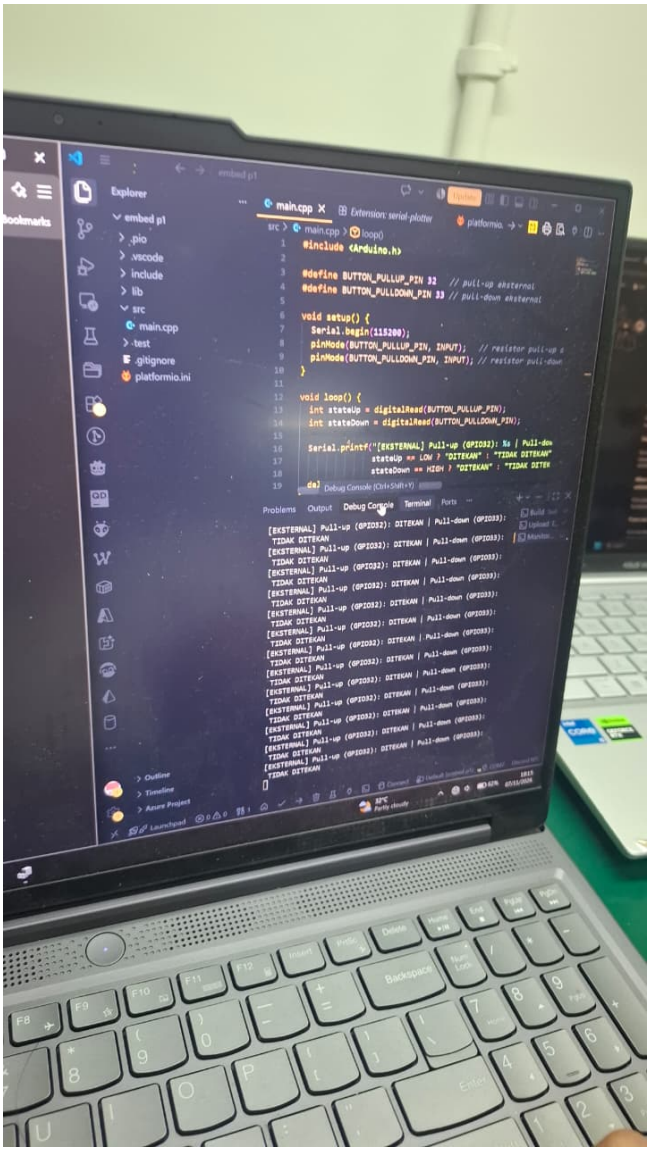
1. Mengganti isi `platformio.ini` menjadi *platform* `espressif32`, *board* `esp32dev`, dan *framework* `arduino`. Berkas `src/main.c` dihapus dan `src/main.cpp` dibuat kembali.
2. Merangkai tombol 1 pada GPIO 32 dengan resistor 10 kΩ ke 3,3 V sebagai *pull-up*, dan kaki tombol lainnya ke GND.
3. Merangkai tombol 2 pada GPIO 33 dengan resistor 10 kΩ ke GND sebagai *pull-down*, dan kaki tombol lainnya ke 3,3 V.



Gambar 7: Merangkai Rangkaian

4. Menulis Kode Program A, yang mengatur kedua pin sebagai `INPUT` biasa dan mencetak status tombol ke *Serial Monitor* setiap 200 ms.

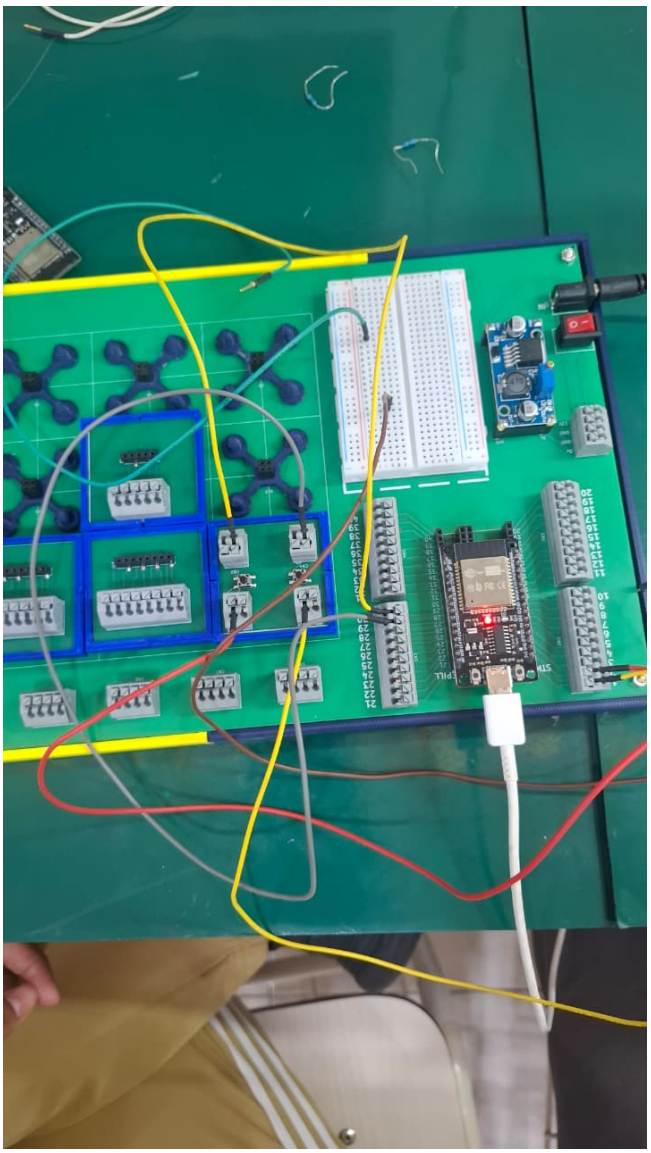
5. Menekan *Build* dan *Upload*, membuka *Serial Monitor* pada 115200 *baud*, menekan kedua tombol, lalu mencatat hasilnya.



Gambar 8: Output Pada Serial Monitor

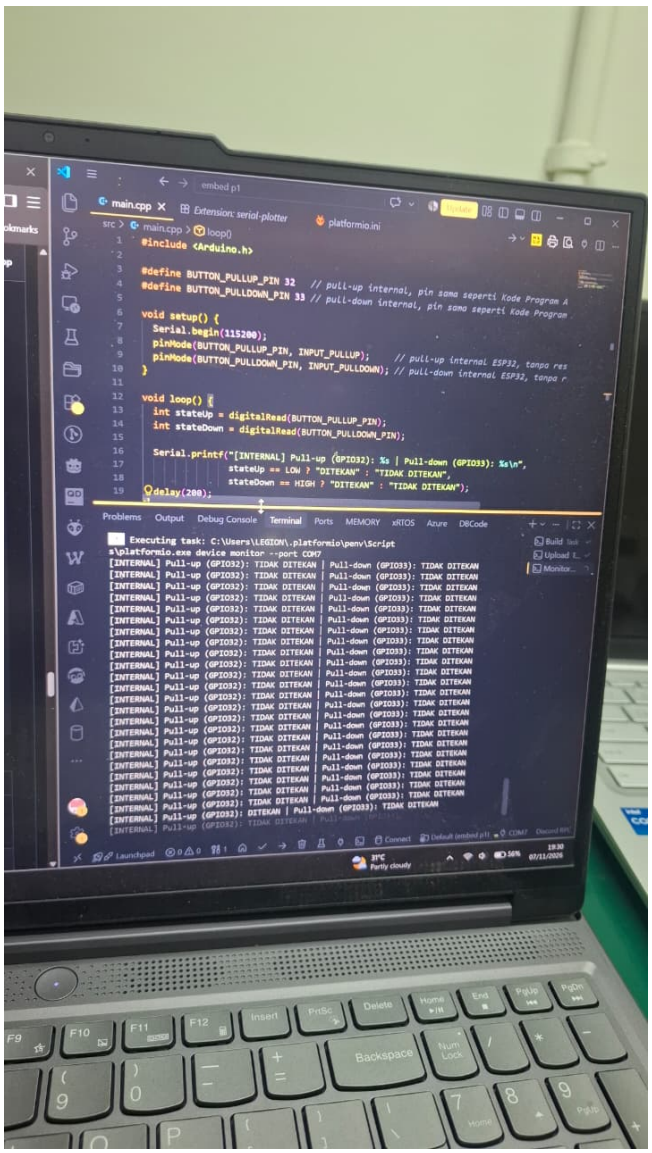
Bagian B: resistor internal

6. Melepas kedua resistor eksternal dengan tombol tetap terpasang pada GPIO 32 (kaki lain ke GND) dan GPIO 33 (kaki lain ke 3,3 V), dengan memastikan pasangan kaki tombol yang dipakai benar.



Gambar 9: Lepas Resistor dalam Rangkaian

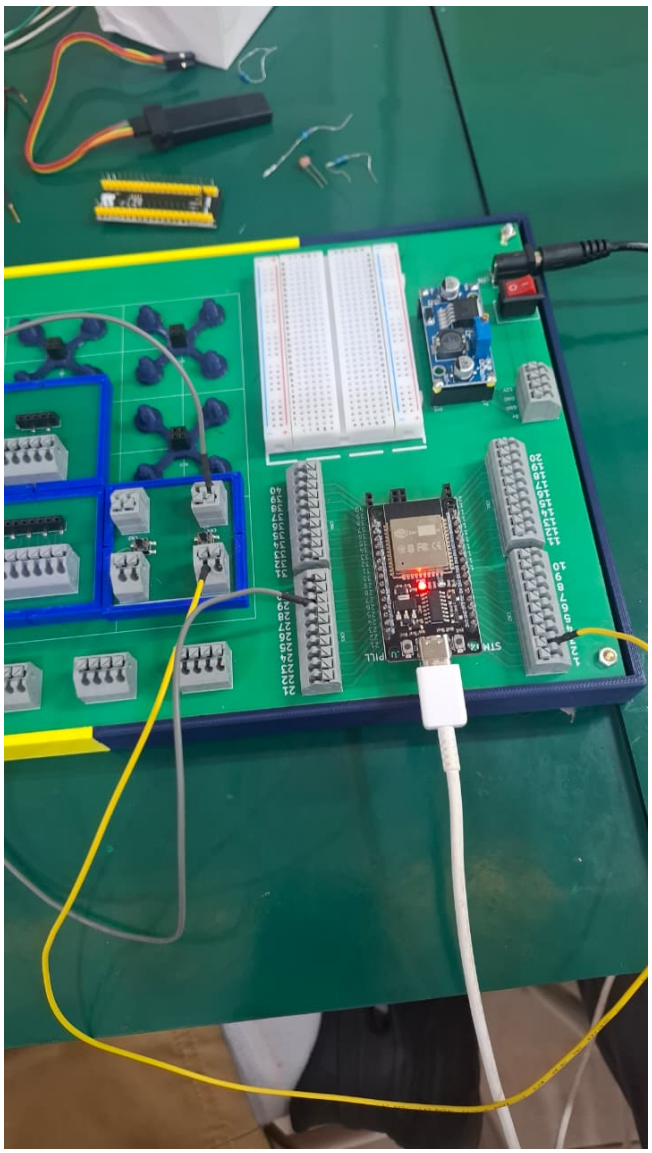
7. Mengganti isi `src/main.cpp` dengan Kode Program B, yang mengatur GPIO 32 sebagai `INPUT_PULLUP` dan GPIO 33 sebagai `INPUT_PULLDOWN`.
8. Menekan *Build* dan *Upload*, menekan kedua tombol yang sama, lalu mencatat hasilnya.
9. Membandingkan hasil Bagian A dan Bagian B. Hasil yang diharapkan sama, yaitu *pull-up* bernilai HIGH saat tidak ditekan dan LOW saat ditekan, sedangkan *pull-down* sebaliknya. Bila hasilnya berbeda, diperiksa apakah kedua resistor eksternal sudah benar-benar dilepas.



Gambar 10: Output Serial Monitor

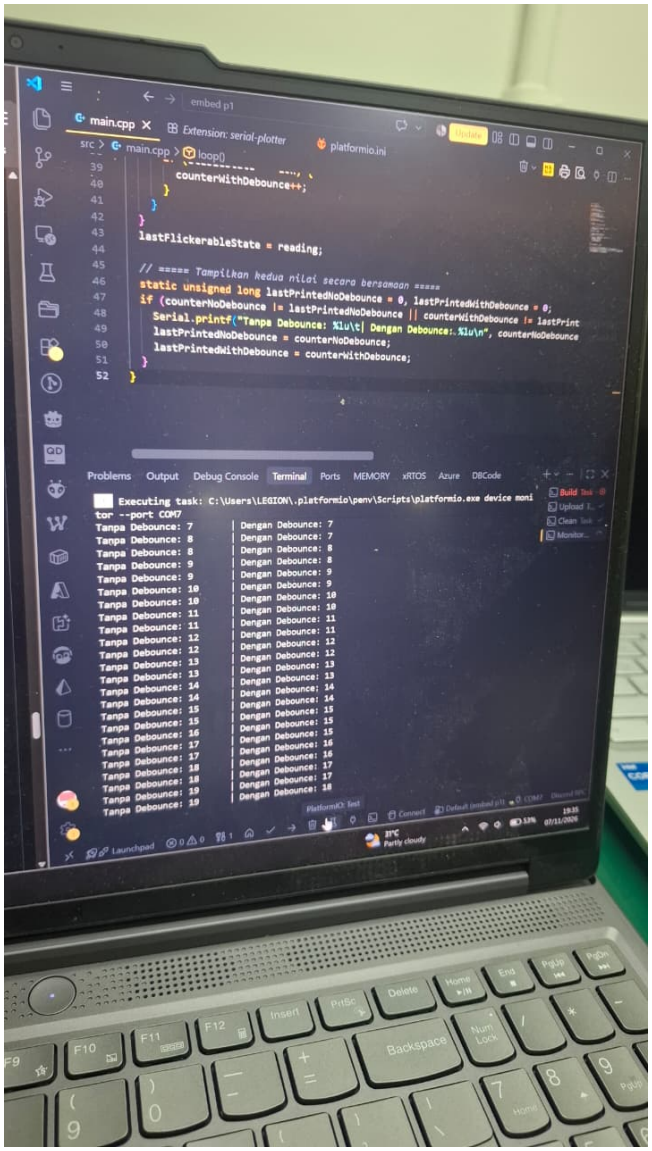
1.4 Percobaan 4: *Debouncing* pada Input Tombol GPIO

1. Memasang tombol pada GPIO 25, dengan satu kaki ke GPIO dan kaki lainnya ke GND. Pin dikonfigurasi dengan `INPUT_PULLUP`. Isi `platformio.ini` sama dengan Percobaan 3.



Gambar 11: Pasang Tombol pada GPIO 25, dengan Satu Kaki ke GPIO dan Kaki Lainnya ke GND

2. Menulis program dengan dua penghitung, yaitu `counterNoDebounce` (tanpa penyaring) dan `counterWithDebo` (dengan penyaring berbasis `millis()`), dengan `DEBOUNCE_DELAY` awal 50 ms.
3. Menekan *Build* dan *Upload*, lalu membuka *Serial Monitor* pada 115200 *baud*.

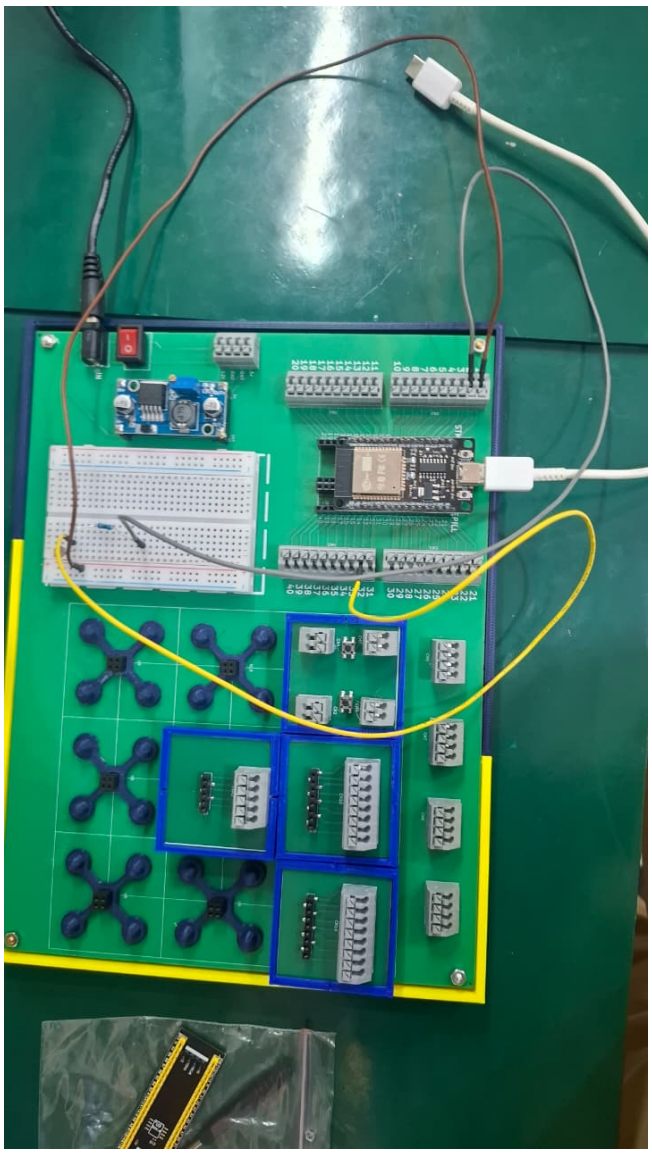


Gambar 12: Build Upload lalu Buka Serial Monitor

4. Menekan tombol satu kali, kemudian membandingkan nilai kedua penghitung.
5. Mengubah `DEBOUNCE_DELAY` menjadi 10 ms, mengunggah ulang program, lalu menekan tombol kembali dan mencatat hasilnya.
6. Mengubah `DEBOUNCE_DELAY` menjadi 200 ms, mengunggah ulang program, lalu menekan tombol dengan cepat berulang kali dan mencatat hasilnya.
7. Membandingkan hasil untuk 10 ms, 50 ms, dan 200 ms, lalu menganalisis kelebihan dan kekurangan tiap nilai.

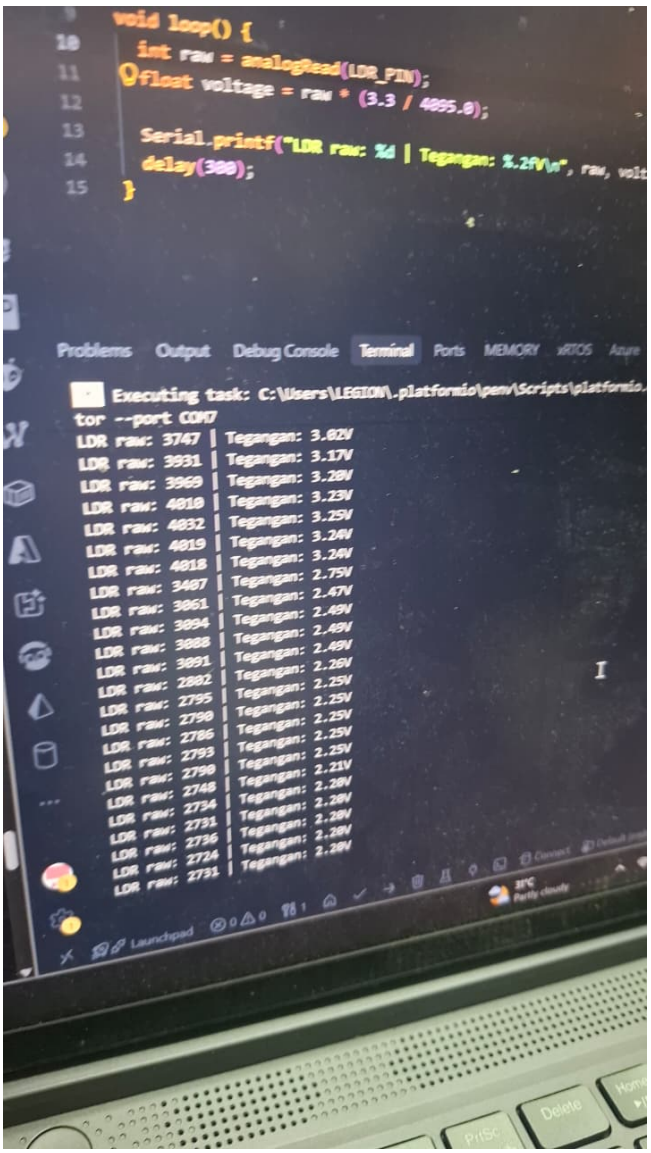
1.5 Percobaan 5: Pembacaan ADC dengan LDR

1. Merangkai pembagi tegangan dengan satu kaki LDR ke 3,3 V dan kaki lainnya ke GPIO 34 serta ke GND melalui resistor 10 kΩ. Isi `platformio.ini` sama dengan Percobaan 3.



Gambar 13: Merangkai Rangkaian

2. Menulis program yang membaca nilai ADC mentah dengan `analogRead()` pada GPIO 34, mengonversinya menjadi tegangan dengan $V = \text{raw} \times 3{,}3/4095$, dan mencetaknya ke *Serial Monitor* setiap 300 ms.
3. Menekan *Build* dan *Upload*, lalu membuka *Serial Monitor* pada 115200 *baud*.
4. Menutup LDR dengan tangan, kemudian menyinarinya dengan lampu, sambil mencatat perubahan nilai mentah (rentang 0 sampai 4095) dan tegangannya.



Gambar 14: Output dari LDR dari Serial Monitor

2 Analisis Hasil Percobaan

Pada Percobaan 1, LED *onboard* PC13 STM32 Blackpill berkedip dengan periode 1 detik (0,5 detik menyala, 0,5 detik padam). Hasil ini sesuai teori karena LED bersifat aktif *LOW*, sehingga menyala saat pin diberi LOW. Unggah melalui ST-Link V2 (SWD) berhasil karena sambungan SWDIO, SWCLK, GND, dan 3,3 V sudah benar. Jika SWDIO dan SWCLK tertukar, ST-Link terdeteksi tetapi *target* tidak terhubung.

Pada Percobaan 2, unggah program ESP-IDF berhasil. ESP32-D0WD-V3 terdeteksi di COM4 dan *baud rate* naik menjadi 460800, sehingga jalur USB-to-Serial (CP2102) bekerja dan tidak diperlukan *programmer* eksternal seperti pada Blackpill. Peringatan *"include errors detected"* berasal dari IntelliSense karena `sdkconfig.h` baru dibuat saat *build*, bukan kesalahan kompilasi. Program berjalan sebagai *task* FreeRTOS lewat `app_main()`, dan `vTaskDelay()` menyerahkan CPU ke *task* lain saat menunggu, berbeda dengan `delay()` pada Arduino.

Pada Percobaan 3 dengan resistor eksternal, *pull-up* (GPIO 32) terbaca *DITEKAN* dan *pull-down* (GPIO 33) terbaca *TIDAK DITEKAN*. Resistor 10 kΩ menarik GPIO 32 ke HIGH dan GPIO 33 ke LOW

saat tombol dilepas, sehingga pembacaan stabil dan tidak *floating*. Pada *pull-up*, tombol yang ditekan menghubungkan pin ke GND sehingga terbaca LOW (*DITEKAN*).

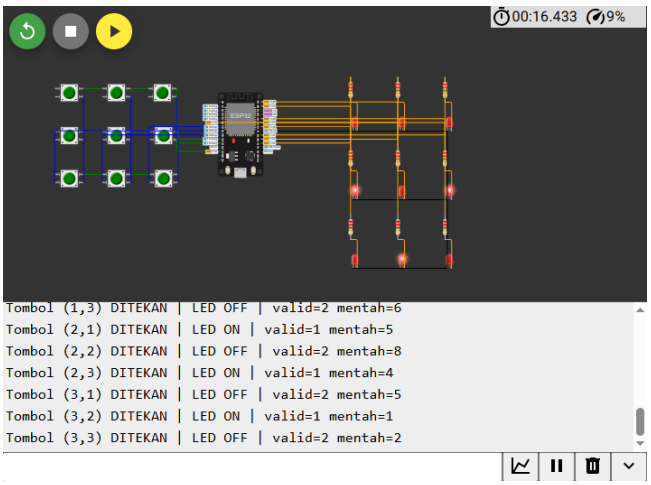
Pada Percobaan 3 tanpa resistor (internal), kedua tombol terbaca *TIDAK DITEKAN* secara stabil walaupun resistor eksternal dilepas, dan *pull-up* berubah menjadi *DITEKAN* saat tombol ditekan. Hal ini menunjukkan `INPUT_PULLUP` dan `INPUT_PULLDOWN` bekerja sama seperti resistor eksternal secara logika. Bedanya, resistor internal ESP32 sekitar 45 kΩ, lebih lemah daripada 10 kΩ sehingga lebih rentan derau, dan tidak tersedia pada GPIO 34–39. Hasil dapat menyimpang jika resistor eksternal belum dilepas atau pasangan kaki tombol salah.

Pada Percobaan 4, `counterNoDebounce` dan `counterWithDebounce` (50 ms) naik dengan jumlah yang sama pada tiap penekanan, sehingga *bouncing* tidak tampak pada tombol yang dipakai. Hal ini wajar karena getaran kontak tidak selalu muncul dan bergantung pada kualitas tombol serta cara menekan. Secara teori, 10 ms responsif tetapi dapat meloloskan *bouncing* yang lebih lama, sedangkan 200 ms bersih tetapi lambat dan dapat melewatkan penekanan cepat. Nilai 50 ms merupakan kompromi yang umum.

Pada Percobaan 5, nilai ADC turun dari sekitar 4030 (3,25 V) ke sekitar 2730 (2,20 V) seiring berkurangnya cahaya. Hasil ini sesuai teori pembagi tegangan, karena makin terang hambatan LDR makin kecil dan tegangan pada GPIO 34 makin besar. Konversinya benar, misalnya $2731 \times 3{,}3/4095 \approx 2{,}20$ V. ADC 12 bit memiliki 4096 tingkat (sekitar 0,8 mV per langkah) sehingga dapat merepresentasikan banyak kondisi, tidak hanya HIGH dan LOW. Nilai maksimum tidak mencapai 4095 karena ADC ESP32 tidak linier di ujung rentang, dan fluktuasi kecil (2790 sampai 2793) berasal dari derau ADC dan cahaya sekitar.

3 Hasil Tugas Modul

Tugas modul berupa simulasi matrix *pushbutton* 3x3 yang mengendalikan matrix LED 3x3 pada ESP32 di Wokwi. Tombol pada posisi (baris, kolom) tertentu mengendalikan LED pada posisi yang sama. Baris tombol dihubungkan ke GPIO 13, 14, dan 27 sebagai *output*, sedangkan kolom tombol dihubungkan ke GPIO 25, 26, dan 33 sebagai *input* dengan `INPUT_PULLUP`. Sembilan LED dihubungkan ke GPIO 4, 16, 17, 18, 19, 21, 22, 23, dan 32, masing-masing melalui resistor 220 Ω. Pembacaan dilakukan dengan memindai baris satu per satu. Hanya baris yang sedang dipindai diberi LOW dan baris lainnya dibiarkan *high-impedance*, sehingga tombol yang ditekan menarik kolomnya ke LOW. Setiap tombol di-*debounce* dengan `millis()` selama 30 ms, sehingga LED baru berubah status setelah pembacaan tombol stabil lebih dari 30 ms. Hasil pada *Serial Monitor* sesuai rancangan. Penekanan tombol (1,1), (1,2), dan (1,3) masing-masing menghasilkan satu pesan tekan dan satu pesan lepas, dan LED pada posisi yang sama (1,1), (1,2), dan (1,3) menyala. Penekanan tombol (3,1) juga hanya menyalakan LED (3,1). Tiap penekanan fisik dicatat sekali oleh program meskipun pembacaan dilakukan berulang kali pada setiap siklus pemindaian, yang menunjukkan bahwa *debouncing* bekerja. Tangkapan layar simulasi ditunjukkan pada Gambar 15, dan *source code* program ditunjukkan pada Kode 1



Gambar 15: Simulasi matrix *pushbutton* 3x3 dan matrix LED 3x3 pada Wokwi

Listing 1: *Source code* matrix *pushbutton* 3x3 dengan *debouncing*

```
#include <Arduino.h>

const int ROWS = 3;
const int COLS = 3;

// Baris = OUTPUT (di-scan), Kolom = INPUT_PULLUP (dibaca)
const int rowPins[ROWS] = {13, 14, 27};
const int colPins[COLS] = {25, 26, 33};

// LED[baris][kolom]
const int ledPins[ROWS][COLS] = {
  { 4, 16, 17},
  {18, 19, 21},
  {22, 23, 32}
};

const unsigned long DEBOUNCE_DELAY = 30; // ms
const bool TOGGLE_MODE = true;

struct Key {
  int lastReading;           // pembacaan mentah terakhir
  int stableState;           // status stabil setelah debounce
  unsigned long lastChange;  // waktu perubahan mentah terakhir
  unsigned long rawEdges;    // jumlah transisi mentah
  unsigned long pressCount;  // jumlah tekanan valid
};

Key keys[ROWS][COLS];
bool ledState[ROWS][COLS];

void setLed(int r, int c, bool on) {
  ledState[r][c] = on;
  digitalWrite(ledPins[r][c], on ? HIGH : LOW);
}

// Hanya baris aktif yang di-LOW-kan, baris lain high-impedance
void selectRow(int active) {
```

```
  for (int r = 0; r < ROWS; r++) {
    if (r == active) {
      pinMode(rowPins[r], OUTPUT);
      digitalWrite(rowPins[r], LOW);
    } else {
      pinMode(rowPins[r], INPUT);
    }
  }
}

void setup() {
  Serial.begin(115200);

  for (int c = 0; c < COLS; c++) pinMode(colPins[c], INPUT_PULLUP);
  for (int r = 0; r < ROWS; r++) {
    pinMode(rowPins[r], INPUT);
    for (int c = 0; c < COLS; c++) {
      pinMode(ledPins[r][c], OUTPUT);
      setLed(r, c, false);
      keys[r][c] = {HIGH, HIGH, 0, 0, 0};
    }
  }
  Serial.println("Matrix pushbutton 3x3 dan matrix LED 3x3 (debounce 30 ms)");
}

void loop() {
  for (int r = 0; r < ROWS; r++) {
    selectRow(r);
    delayMicroseconds(50); // beri waktu sinyal stabil

    for (int c = 0; c < COLS; c++) {
      Key& k = keys[r][c];
      int reading = digitalRead(colPins[c]); // LOW = ditekan

      // Transisi mentah HIGH ke LOW (tanpa debounce)
      if (reading == LOW && k.lastReading == HIGH) k.rawEdges++;

      // Catat waktu setiap perubahan pembacaan mentah
      if (reading != k.lastReading) k.lastChange = millis();

      // Terima perubahan hanya jika sinyal stabil > DEBOUNCE_DELAY
      if ((millis() - k.lastChange) > DEBOUNCE_DELAY && reading != k.stableState) {
        k.stableState = reading;

        if (k.stableState == LOW) {            // tekanan valid
          k.pressCount++;
          if (TOGGLE_MODE) setLed(r, c, !ledState[r][c]);
          else setLed(r, c, true);
          Serial.printf("Tombol (%d,%d) DITEKAN | LED %s | valid=%lu mentah=%lu\n",
                        r + 1, c + 1, ledState[r][c] ? "ON" : "OFF",
                        k.pressCount, k.rawEdges);
        } else if (!TOGGLE_MODE) {             // tombol dilepas
          setLed(r, c, false);
        }
```

```
92        }
93        k.lastReading = reading;
94      }
95    }
96  }
```

Program menyimpan status tiap tombol dalam struktur `Key` yang berisi pembacaan mentah terakhir (`lastReading`), status stabil (`stableState`), waktu perubahan terakhir (`lastChange`), serta penghitung transisi mentah dan tekanan valid. Pada `loop()`, setiap baris diaktifkan bergantian, lalu ketiga kolomnya dibaca. Status stabil hanya diperbarui bila pembacaan tidak berubah selama lebih dari `DEBOUNCE_DELAY`, sehingga LED tidak mengikuti setiap perubahan sinyal mentah. Pada mode *toggle*, setiap tekanan valid membalik status LED yang bersesuaian.

1. *Debouncing* tetap diperlukan karena bouncing terjadi pada kontak mekanis tiap tombol, bukan karena jumlah tombol yang ditekan. Saat tombol ditekan atau dilepas, kontak logam di dalamnya belum langsung menyatu atau terpisah dengan tenang, melainkan memantul selama beberapa milidetik. Akibatnya sinyal berubah HIGH dan LOW berulang kali sebelum stabil. Faktor penyebabnya adalah pegas dan kontak logam yang bergetar, permukaan kontak yang tidak rata, serta kekuatan dan kecepatan penekanan. Pada matrix, setiap tombol dibaca berulang kali dalam tiap siklus pemindaian yang berlangsung sangat cepat, jauh lebih singkat dari durasi bouncing. Satu tekanan fisik pun dapat terbaca sebagai beberapa tekanan, sehingga pada mode *toggle* LED berubah status berkali-kali dan keadaan akhirnya tidak dapat diprediksi. Karena itu syarat satu tombol per waktu tidak menghilangkan masalah ini. Penyaringan 30 ms membuat perubahan hanya diterima setelah sinyal stabil.

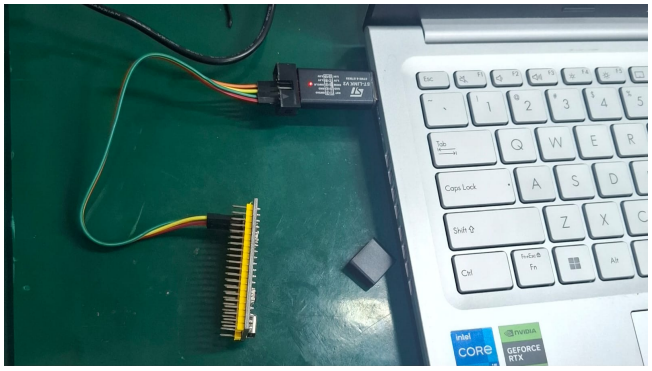
2. Pembacaan kolom memakai konfigurasi *pull-up*, yaitu `INPUT_PULLUP` yang mengaktifkan resistor internal ESP32 (sekitar 45 kΩ) ke 3,3 V. Saat tombol tidak ditekan, pin kolom terbaca HIGH karena ditarik ke 3,3 V oleh resistor tersebut. Saat tombol ditekan, kolom terhubung ke baris yang sedang aktif (LOW), sehingga pin terbaca LOW dan program menafsirkannya sebagai tombol ditekan. Pin tidak boleh dibiarkan *floating* karena pin yang tidak terhubung ke level logika apa pun akan mengambang pada tegangan yang tidak menentu dan mudah dipengaruhi derau serta medan listrik di sekitarnya. Pembacaannya dapat berubah acak antara HIGH dan LOW, sehingga program menganggap ada tombol ditekan padahal tidak. Resistor *pull-up* menjamin kolom selalu berada pada level HIGH yang pasti saat tombol dilepas.

3. Pada masukan digital, pin hanya membedakan dua kondisi menurut ambang tegangannya. Pada ESP32 tegangan di atas sekitar $0{,}75\,V_{DD}$ dibaca HIGH, tegangan di bawah sekitar $0{,}25\,V_{DD}$ dibaca LOW, dan tegangan di antaranya tidak terdefinisi. Informasi tentang besar tegangan hilang, sehingga cocok untuk tombol yang hanya memiliki kondisi tekan dan lepas. Pada masukan analog, tegangan dikonversi oleh ADC menjadi angka sehingga besarnya tetap terbaca. ADC 12 bit pada ESP32 membagi rentang 0–3,3 V menjadi $2^{12} = 4096$ tingkat (0–4095), dengan resolusi sekitar 0,8 mV per tingkat. Nilai ini dihitung dengan $V = \text{raw} \times 3{,}3/4095$, sehingga satu pin dapat merepresentasikan ribuan kondisi, bukan hanya dua. Bila salah satu tombol diganti LDR, LDR dipasang sebagai pembagi tegangan dengan resistor 10 kΩ dan keluarannya dibaca dengan `analogRead()` pada pin ADC yang mendukungnya, misalnya GPIO 34. Makin terang cahaya, makin kecil hambatan LDR dan makin besar tegangan pin, sehingga intensitas cahaya

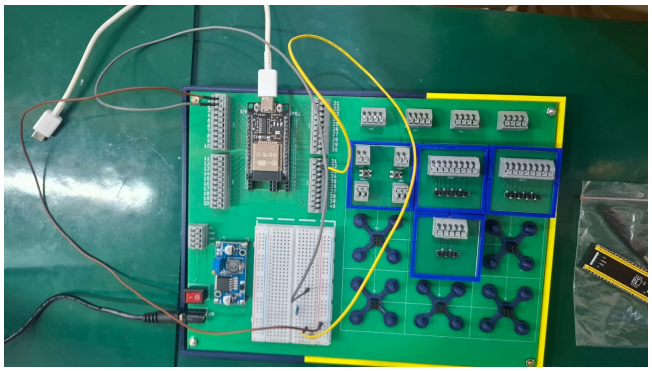
dapat dibedakan secara bertahap. Pin ini tidak dapat dipakai sebagai kolom matrix, karena resistor *pull-up* dan pemindaian baris akan mengganggu pembagi tegangan.

4 Lampiran

4.1 Dokumentasi saat praktikum



Gambar 16: Dokumentasi 1



Gambar 17: Dokumentasi 2